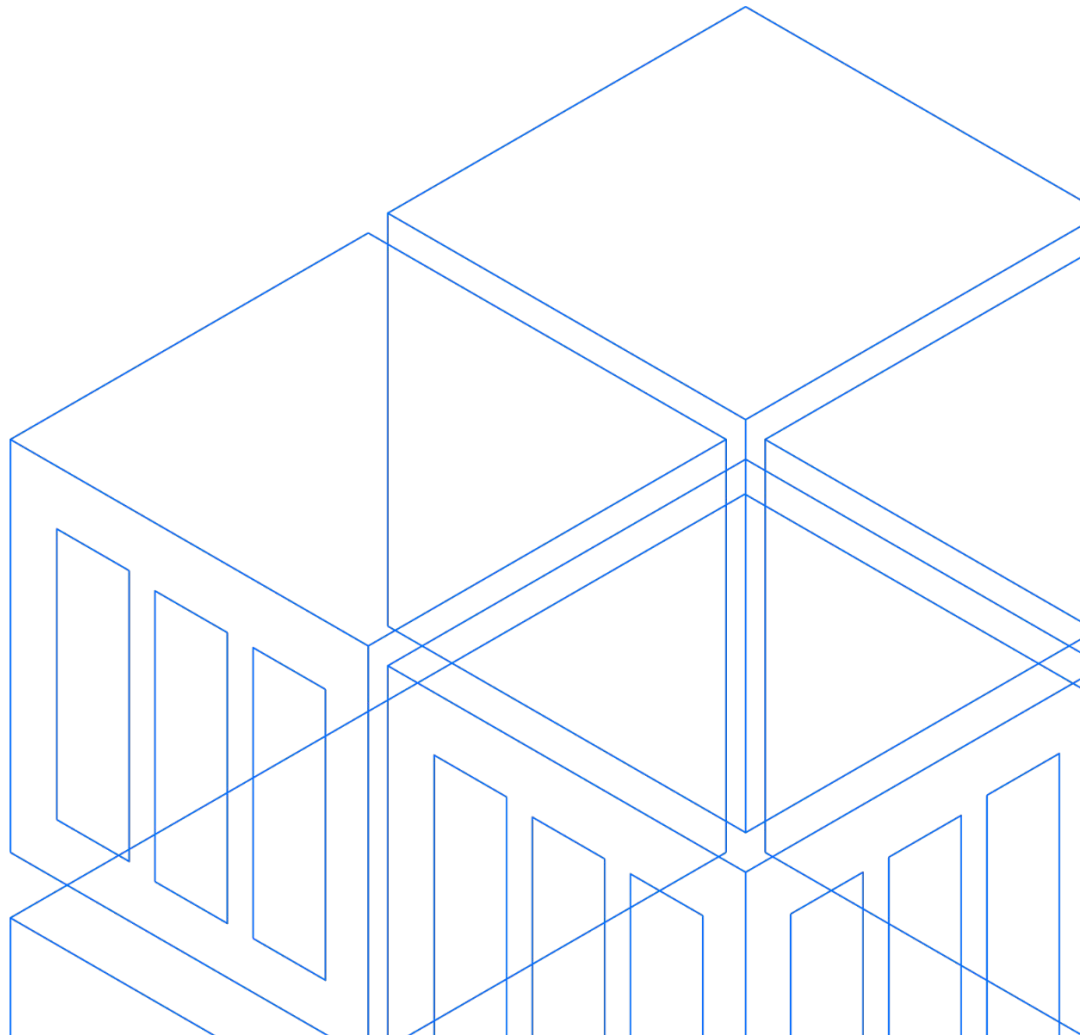




FM 4 Cert (w/ AI)

Dr. Gregory Malecha
Director of Formal Methods
gregory@bluerock.io





Agenda

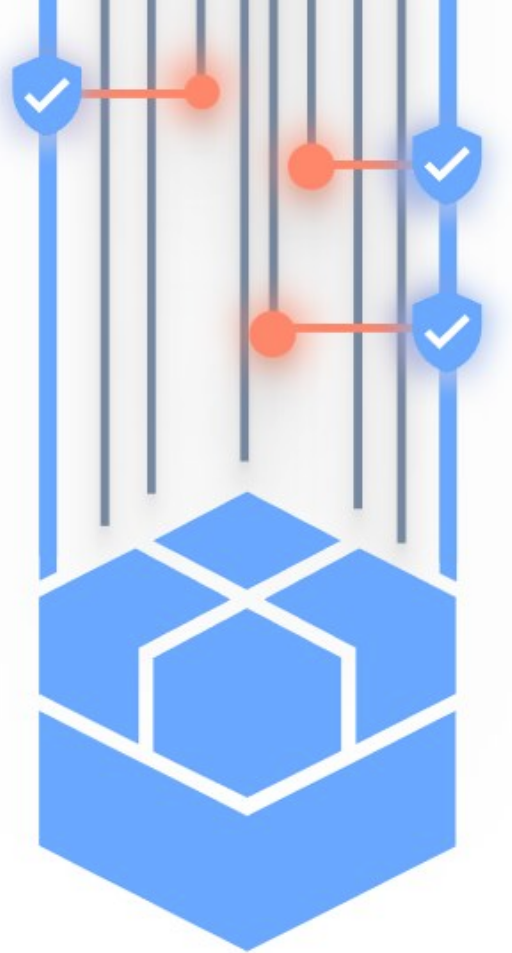
1. Certification with Proofs
2. FM / AI

BlueRock at a Glance

Secure systems from entire classes of attacks.

- Application runtime security.
- Container Security
- BlueRock ULTRA – virtualization platform for cloud and embedded environments.
 - DARPA SBIR w/ NavAir. (PM. Lok Yan)
 - Formal verification
- Verification tools focused on systems code.

<https://bluerock.io/>





Proof-based Certification

The \$\$ Value of a Proof

Don't do more, do it more efficiently.
"What does a proof replace?"

Safety vs Security

Safety "for a specific purpose"

- Strong assumptions about use and operating environment.
- Could be abused by malicious actors.

Less relevant in a highly connected situations.

Security "against malicious agents"

- Weak assumptions about the environment (actively malicious).
- Requires building an attacker model.

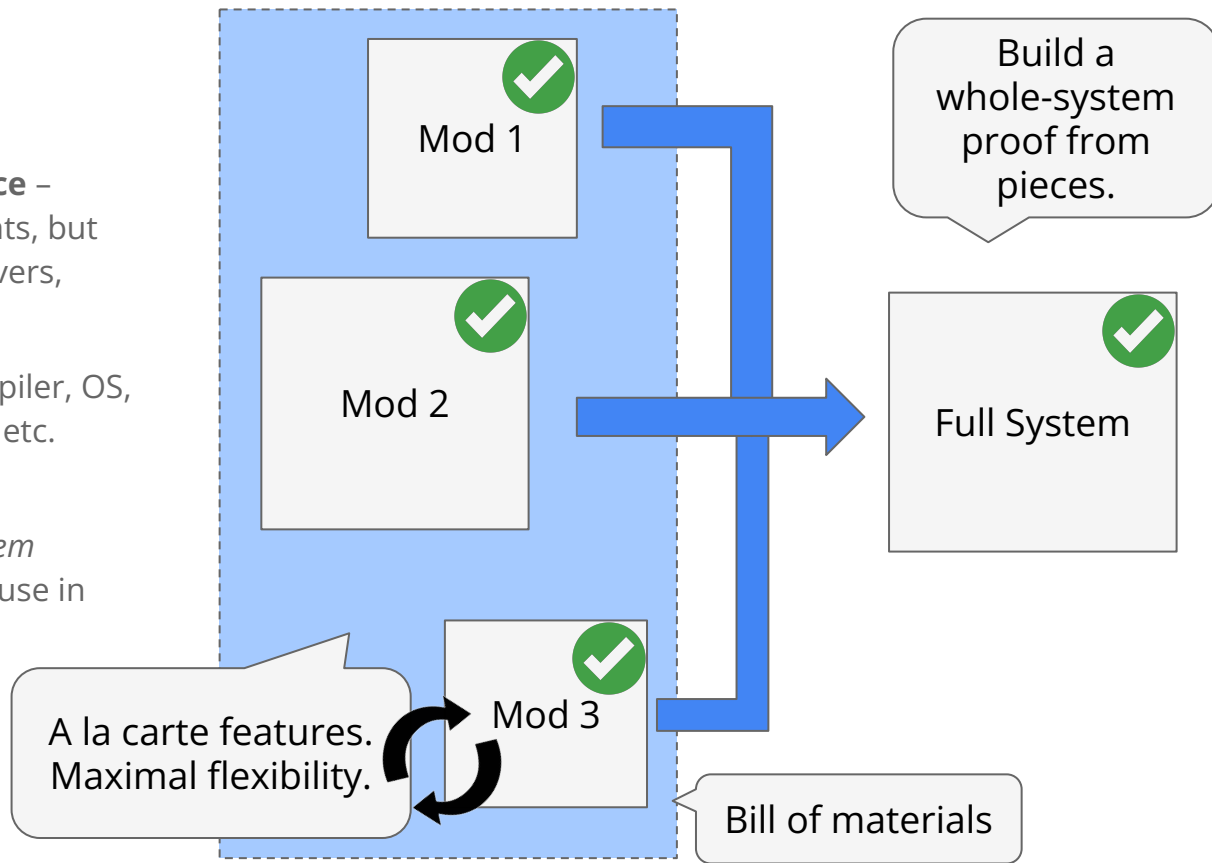
Anticipate and protect against potential attacks, or prove the absence of exploits against all possible attackers.

...and re-certification

Compositional Certification w/ Proofs

Proofs provide

- **Machine-checkable Evidence** – especially with proof assistants, but also other tools, e.g. SMT solvers, model checkers, etc.
- **Explicit Assumptions** – compiler, OS, environmental assumptions, etc.
- **Hypothetical Reasoning** – Potentially enables *open system* certification. "Safe/secure to use in any context satisfying..."



What Properties Should we Focus on?

Challenge #1

What system properties should we focus on first?

- **Safety properties** – behavioral refinements, functional correctness, timing guarantees, etc.
- **Hyper properties** – non-interference
- **Side-channel Properties** – often not explicit in the model, often unwieldy to reason about, e.g. "universal composability".

What is a Proof?

Challenge #2

Don't trust people, trust proofs.

- Need an expressive foundation for proofs.
 - Proof-assistants seem to be the obvious choice here.
- Enable innovation (market forces) in terms of how to produce proofs.
- Do we need to develop separate proof checkers, e.g. like coqchk?
 - 2nd, independent implementation

Can it ALL be Proved?

Challenge #3

If proofs are so great, why not prove everything?

Gut reaction: "It's great that you're doing proofs, but you haven't proven everything (yet)."

Incrementality is crucial.

- Partially verification > no verification
- Demonstrate feasibility & cost savings in some modules, replicate in other modules.

Evaluation

Develop a **proof-based evaluation methodology and trial it** to develop a real system.

- Reproducible methodology with machine-checked artifacts.
- Combine results from multiple contributors.
- Connections between different layers, e.g. OS, compilers, applications.

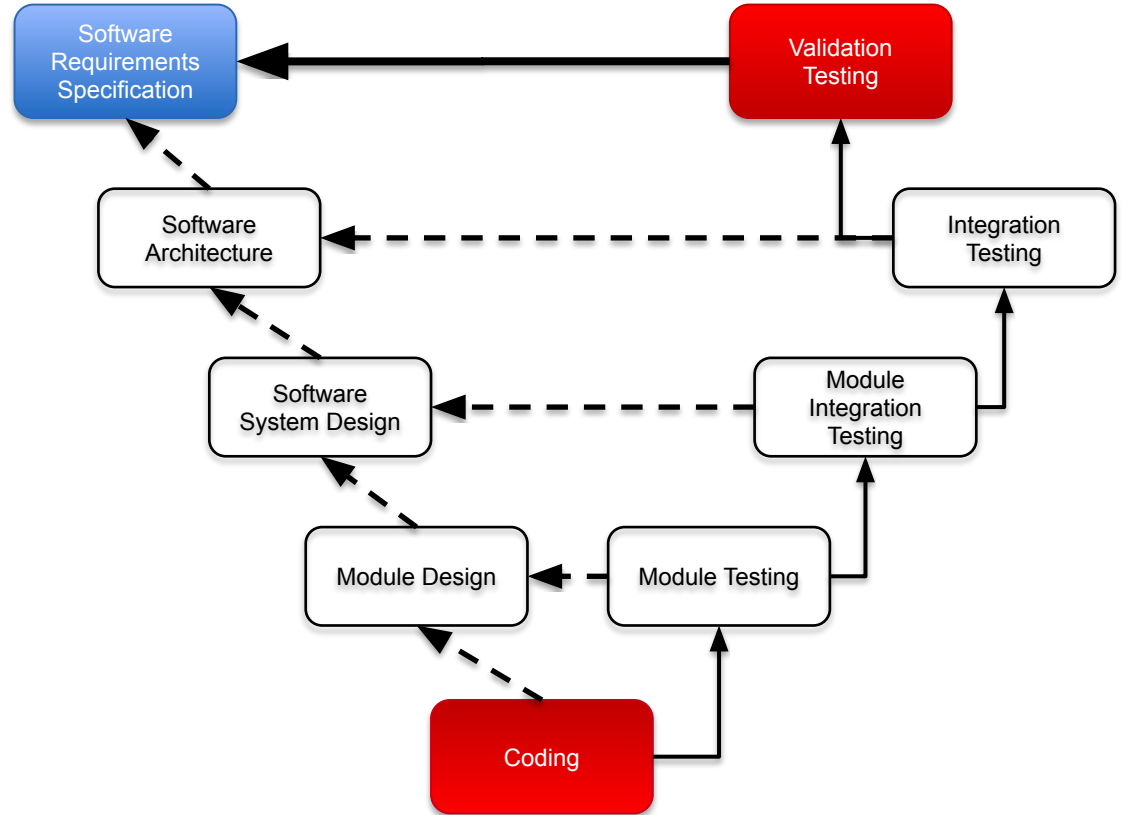
Milestones: If the system says "yes", these requirements are satisfied.

Ideal: If the system says "yes", it is certified.

Potential Valuable Partners

- "3-letter" agencies
- NIST
- UK GCHQ, NCSC
- VeTTS

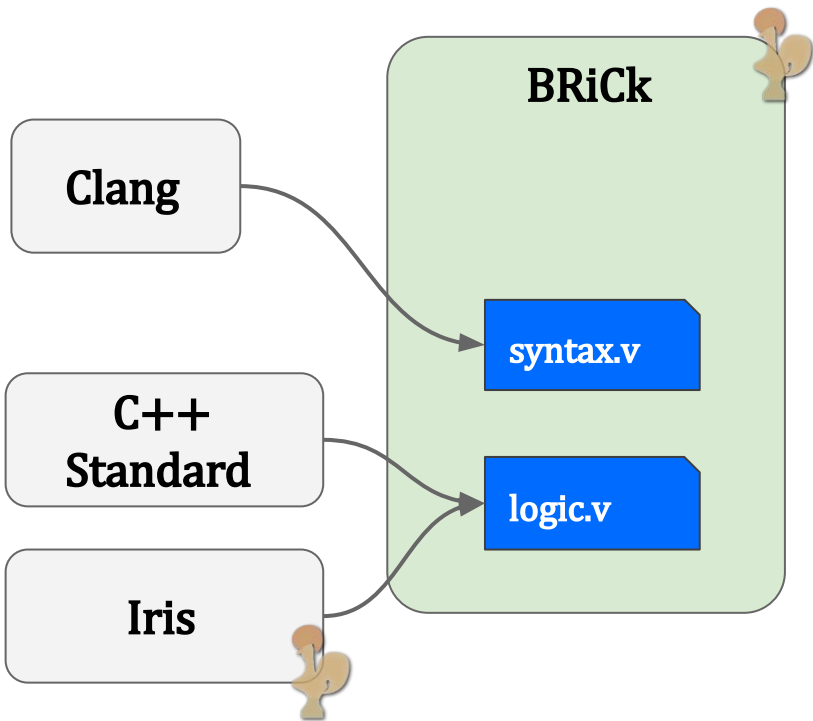
Accelerating the V-Model





BRiCk C++ Infrastructure

<https://github.com/bluerock-io/brick>



Supports a large fragment of modern C++.

- Classes & inheritance
- auto
- Operator overloading
- Virtual dispatch
- Atomics
- new & delete
- Destructuring declarations
- Closures
- Templated code (post-instantiation)

More support driven by user needs:

- Exceptions?

BRiCK C++ Infrastruct

<https://github.com/bluerock-io/brick>

BRiCK

Separation logic-based semantics using Iris.

- Covers behaviors such strict aliasing,
- Separation of the semantics and the automation.

Iris

```
(** [Cl2r] represents reads of locations.  
This counts as an _access_, so it must happen at one of the types listed in  
https://eel.is/c++draft/basic.lval#11.  
*)  
Axiom wp_operand_cast_l2r :  $\forall e Q,$   
  (letI* a, free := wp_glval e in  
     $\exists v,$   
    ( $\exists q, a \mapsto$  tptsto_fuzzyR (erase_qualifiers $ type_of e) q v ** T)  $\wedge$   
    Q v free)  
   $\vdash$  wp_operand (Ecast Cl2r e) Q.
```

Program state reflected in separation logic.

```
(** The '*' operator is an lvalue, but this does not perform an  
access (see [wp_operand_cast_l2r] instead).  
  
We check pointer [p] is strictly valid and aligned.  
The standard says (<https://eel.is/c++draft/expr.unary.op#1>):  
  
> The unary * operator performs indirection: the expression  
> to which it is applied shall be a pointer to an object type,  
> or a pointer to a function type and the result is an lvalue  
> referring to the object or function to which the expression  
> points. If the type of the expression is "pointer to T", the  
> type of the result is "T".  
  
*)  
Axiom wp_lval_deref :  $\forall ty e Q,$   
  letI* v , free := wp_operand e in  
   $\exists p, [v = \text{Vptr } p] \text{ **}$   
  reference_to (erase_qualifiers ty) p **  
  Q p free  
   $\vdash$  wp_lval (Ederef e ty) Q.
```

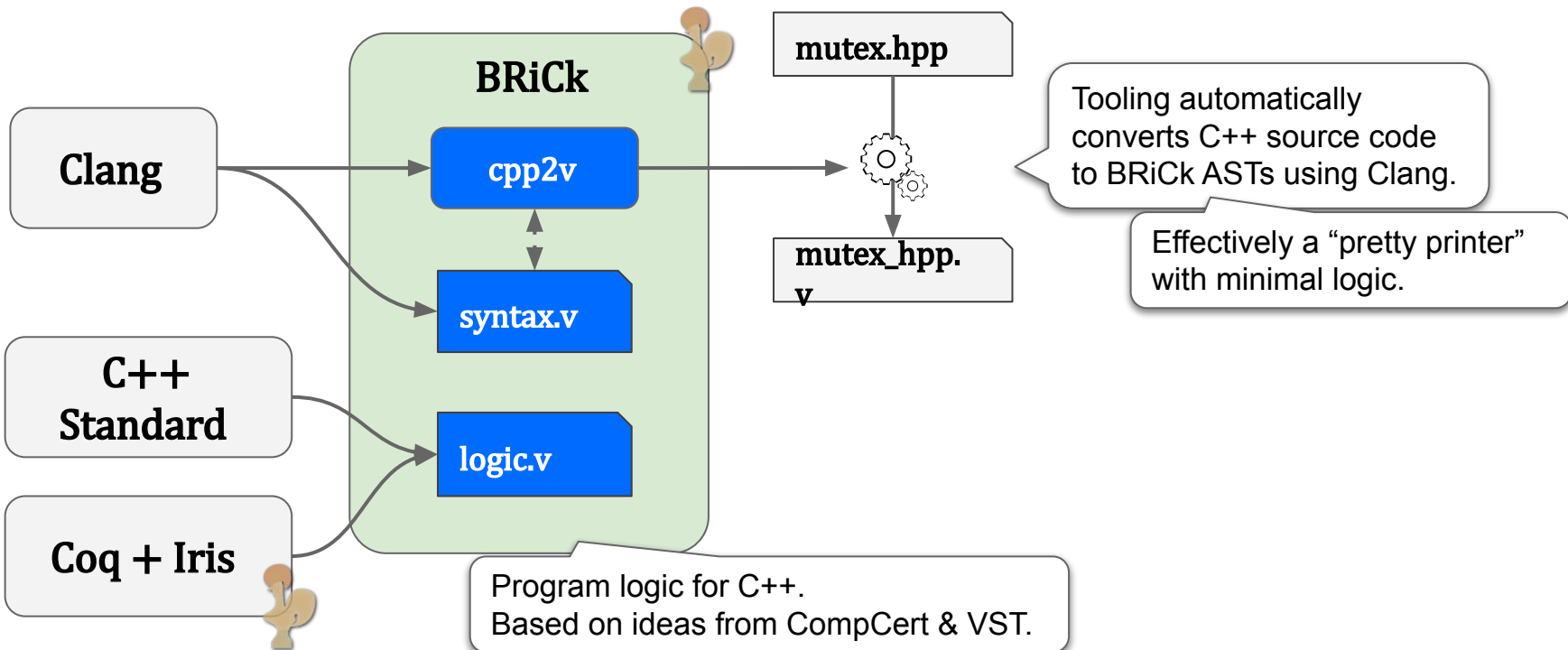
Object lifetime and
bounds checking

Evaluation order

```
(** Evaluation of binary operators. *)  
Axiom wp_operand_binop :  $\forall o e_1 e_2 ty Q,$   
  letI* '(v1,v2), free := nd_seq (wp_operand e1) (wp_operand e2) in  
   $\exists v',$   
  (eval_binop tu o (type_of e1) (type_of e2) ty v1 v2 v' ** T)  $\wedge$   
  Q v' free)  
   $\vdash$  wp_operand (Ebinop o e1 e2 ty) Q.
```

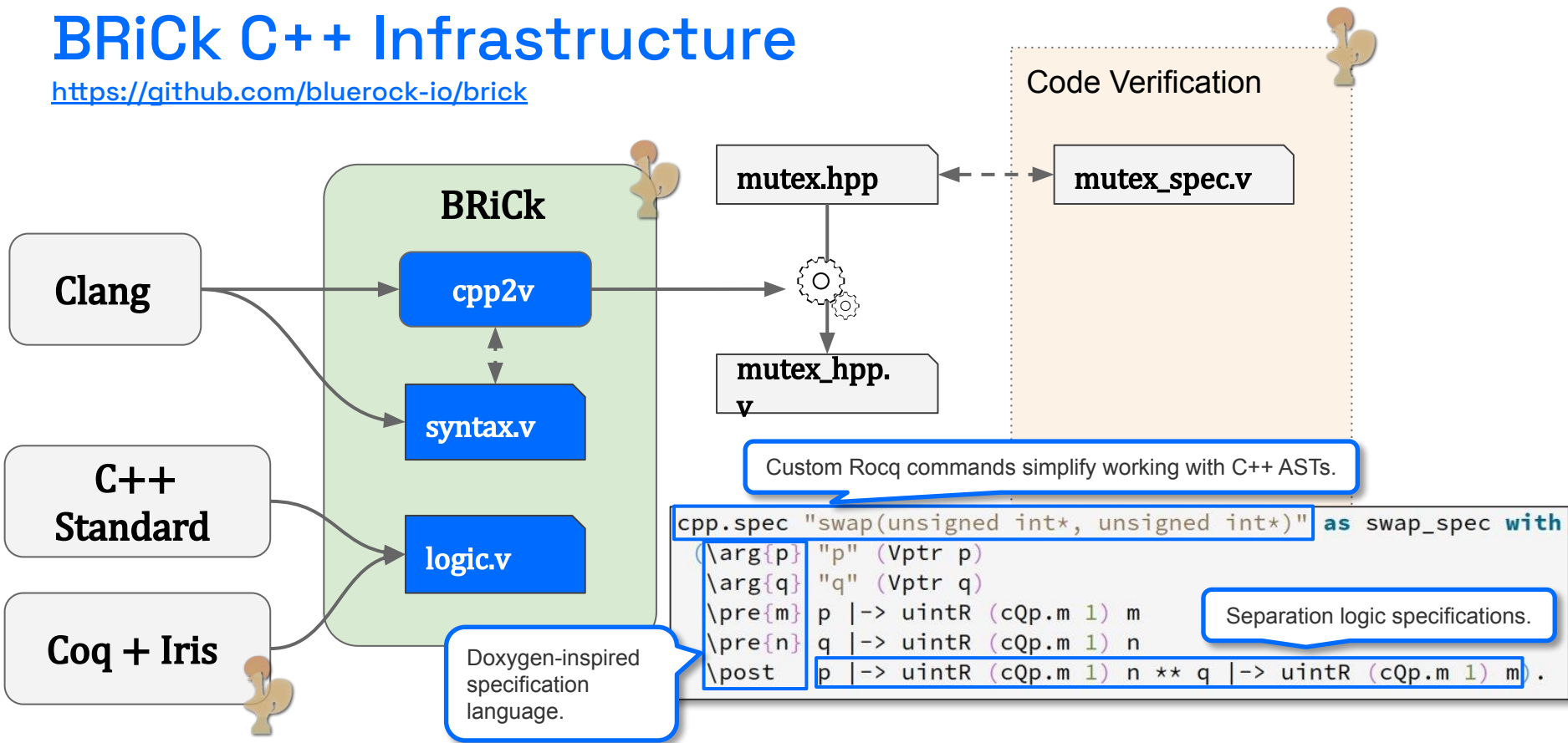
BRiCk C++ Infrastructure

<https://github.com/bluerock-io/brick>



BRiCk C++ Infrastructure

<https://github.com/bluerock-io/brick>



Mutex Specifications

Specification captures the ownership transfer.

- Mutex protects an invariant.
- **enter()** transfers resources to the caller.
- **exit()** transfers resources to the mutex.

Sequential ownership

```
cpp.spec "Sys::Mutex::enter()" as enter
  with (λ (this : ptr)
    \nova pd ec
    \persist{γ P} mutex.I (Some γ) P
    \prepost{q} this |-> mutex.R pd (Some γ) q
    \pre mutex.contender γ 1
    \post[Vbool true] P ** mutex.locked γ).
    Trade contender for lock ownership

cpp.spec "Sys::Mutex::exit()" as exit
  with (λ (this : ptr) =>
    \nova pd ec
    \persist{γ P} mutex.I (Some γ) P
    \prepost{q} this |-> mutex.R pd (Some γ) q
    \pre |> P ** mutex.locked γ
    \post[Vbool true] mutex.contender γ 1).
    Return lock resources to get contender
```

User-defined representation predicates encapsulate implementation details.

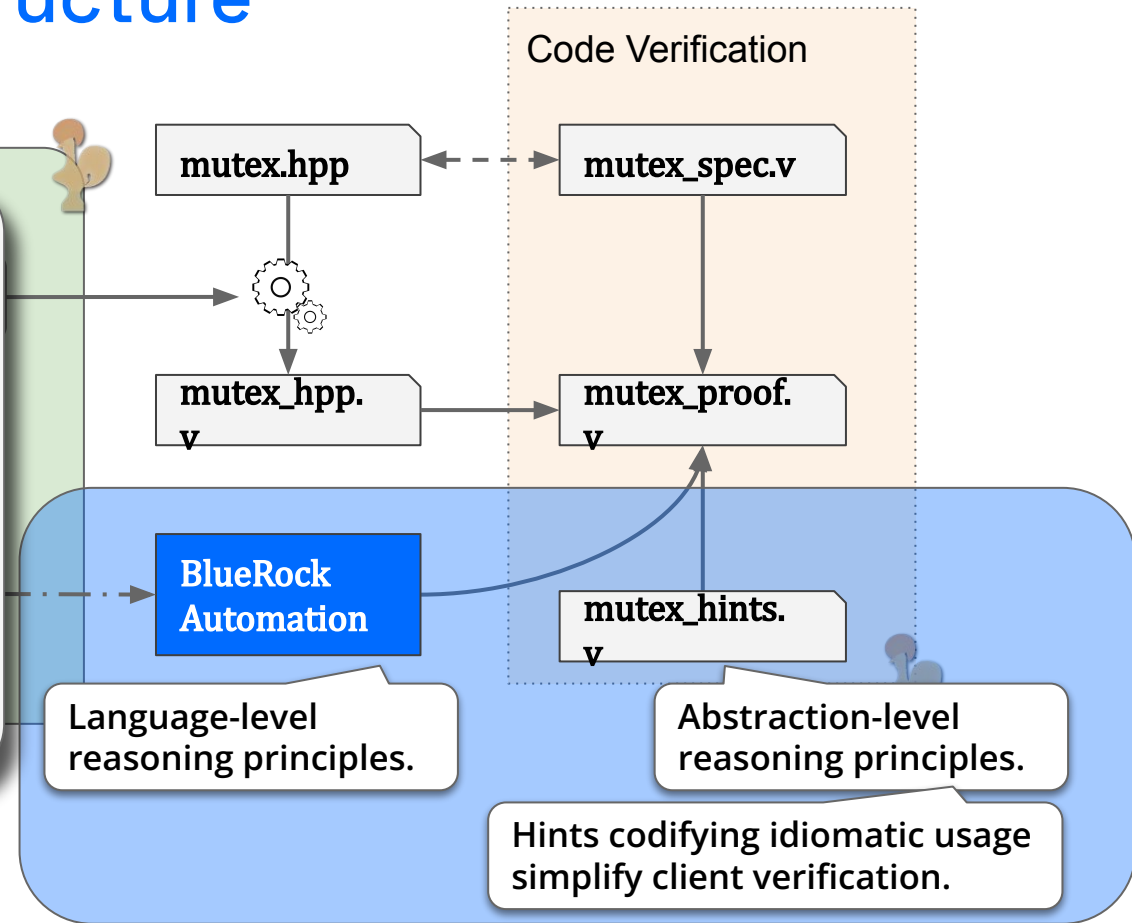
Unrestricted use of Iris features for reasoning.

BRiCK C++ Infrastructure

<https://github.com/bluerock-io/brick>

Generic, **extensible** separation logic automation enables:

- Library-specific automation to capture idiomatic usage.
- Flexible reasoning about non-language constructs, e.g. OS abstractions.
- Rapid prototyping for new generic reasoning principles, e.g. concurrency, sharing, etc.
- Easy support of new language features.



Verification with BRiCK

Language-level Reasoning

Hints capture reasoning principles for the automation to [apply automatically](#).

- “Forward” hints work on premises
- “Backward” hints work in the conclusion
- “Cancellation” hints connect premises and conclusions.
- “Learnable” hints learn pure facts from conjunctions.

```
Axiom wp_lval_deref : ∀ ty e Q,  
  letI* v , free := wp_operand e in  
  ∃ p, [| v = Vptr p |] **  
    reference_to (erase_qualifiers ty) p **  
    Q p free  
  ⊢ wp_lval (Ederef e ty) Q.
```

“One-liners”
convert lemmas
into a form usable
by the core
automation.

Definition `wp_operand_addr_of_B` := [BWD] @wp_operand_addr_of.

**BlueRock
Automation**

Extend the automation with
BRiCK’s language-level
reasoning principles.

Verification with BRiCK

Program Abstractions

The same hint infrastructure is used to reason about program-specific abstractions.

- Especially useful for justifying **concurrency protocols**.
- Program libraries + hints covering idiomatic usage simplifies client verification.

Domain experts can provide abstractions with hints.

```
Lemma do_acquire γ Q P q :  
  cinv mutex.NS γ.(lg_inv) (Inv γ P) ** cinv_own γ.(lg_inv) q  
  ⊢ [| Q = |> (P ** locked γ) ** cinv_own γ.(lg_inv) q |]  
  -* semaphore.do_acquire γ.(sm_ghost) Q.
```

New, generic automation is making some of these **concurrency hints unnecessary**.

```
Definition do_release_C := [CANCEL] do_release.  
  
Theorem enter_ok :  
  denoteModule mutex_hpp.module  
  ⊢ semaphore.acquire -* mutex.enter.  
Proof. verify_spec; rewrite /mutexR; ego. Qed.  
  
Theorem exit_ok :  
  denoteModule mutex_hpp.module  
  ⊢ semaphore.release -* mutex.exit.  
Proof. verify_spec; rewrite /mutexR/locked; ego. Qed.
```

Automation is **constantly** improving.

Verification with BRiCK

Reusable hints and hint infrastructure is crucial to taming the verbosity of low-level code.

- Reduce overhead for “follow your nose” proofs.
- Make it tractable for non-experts to **maintain** & (sometimes) **develop** complex proofs.

```
Lemma create_pd_raw2safe_ipm : spec_entails raw.create_pd_spec safe.create_pd_spec.  
Proof.  
  iIntros (vs K) * /=.  
  iDestruct 1 as  
    (y_caller_ec y_caller_obj y_caller_hst op sel_target sel_owner_pd y_owner_pd_pd_rights q_pd ty) "R".  
  iDestruct "R" as (+) "(ECobj & Echst & HpdRights & CapOwn & CapTgt & %tyS & SP & Tail)".  
  iExists _, _, _, (λ s, K (status.to_V s)), _, _, iFrame "ECobj Echst".  
  iSplit; first done.  
  iSplit; last by iIntros (?) "S".  
  iApply (resolve_sel_rights_intro_weak_sequential HpdRights with "CapOwn [-]"); iIntros "CapOwn /=".  
  rewrite tyS.  
  rewrite /create_pd.create_obj /create_pd.create_hst  
    /create_pd.create_first_space /create_pd.create_pio /=.  
  destruct ty.  
  1, 4, 5 : (== PD, GST, DMA *)  
    iApply (create_sel_intro_weak_sequential with "CapTgt");  
    repeat iSplit; iIntros; iApply "Tail" * /=; eauto with iFrame.  
  1, 2 : (== OBJ, HST *)  
    iApply (do_reserve_spc_success_intro_weak_sequential with "SP");  
    iIntros "SP";  
    iApply (create_sel_cap_intro_weak_sequential with "CapTgt");  
    (repeat iSplit; [...iIntros (v_spc)]; iIntros "Cap";  
    iApply (do_reserve_spc_commit_or_cancel_intro_weak_sequential with "SP");  
    iIntros "SP"; iApply "Tail" * /=; eauto with iFrame).  
  (** PIO *)  
  - iDestruct (x86_only_elim with "SP") as (IS_x86 v_pio v_hst) "[SP SH]".  
    iApply (do_reserve_spc_failure_intro_weak_sequential with "SP").  
    iIntros "SP".  
    iApply x86_only_intro; [done].  
    iApply (do_read_pd_hst_intro_weak_sequential with "SH"). iIntros "SH".  
    iApply (create_sel_cap_intro_weak_sequential with "CapTgt").  
    repeat iSplit; [...iIntros (v_spc)]; iIntros "Cap";  
    iApply "Tail" * /=;  
    rewrite -(x86_only_intro _ _ _ IS_x86);  
    eauto 10 with iFrame.  
  (** MSR *)  
  - iDestruct (x86_only_elim with "SP") as (IS_x86) "_".  
    iApply x86_only_intro; [done].  
    iApply (create_sel_intro_weak_sequential with "CapTgt");  
    repeat iSplit; iIntros; iApply "Tail" * /=;  
    rewrite -(x86_only_intro _ _ _ IS_x86);  
    eauto with iFrame.  
Qed.
```



Several **reusable** hints later...

```
Lemma create_pd_raw2safe : spec_entails raw.create_pd_spec safe.create_pd_spec.  
Proof.  
  iIntros (vs K) "/*".  
  rewrite /create_pd.spec /create_pd.success  
    /create_pd.create_obj /create_pd.create_hst  
    /create_pd.create_first_space /create_pd.create_pio /=.  
  work.  
  ren_hyp ty spc.type.t.  
  ren_hyp tyS (spc.type.of_N _ = Some ty).  
  rewrite tyS.  
  case_match; work.  
  all: eauto.  
Qed.
```

Verification with BRiCK

Accelerate the Simple

Code generation reduces the burden of low-level code.

- Synthesize **functional models and class invariants** for simple classes.
- Synthesize simple hints.
- Synthesize **specification and proofs** for simple: constructors, destructors, and assignment operators.

Synthesized specifications integrate seamlessly with manual specifications.

Simple 2-d point class, easy to specify by hand.

```
(** a simple [Rep]resentation predicate (aka class invariant) for  
  <<struct point>> *)  
br.lock  
Definition pointR `{Σ : cpp_logic} {σ : genv} (q : cQp.t) (m : Z * Z) :=  
  structR ``::point`` q **  
  _field ``::point::x`` |-> intR q (fst m) **  
  _field ``::point::y`` |-> intR q (snd m).  
#[only(lazy_unfold)] derive pointR.
```

Automatically derive some infrastructure hints.

Even more aggressive infrastructure can generate this and more.

```
Module point.  
  cpp.class "point" prefix "" from module dataclass.  
End point.
```

Efficiency improvements enable us to focus on the hard parts of the verification.



Industrial Usage Verifying BlueRock ULTRA™

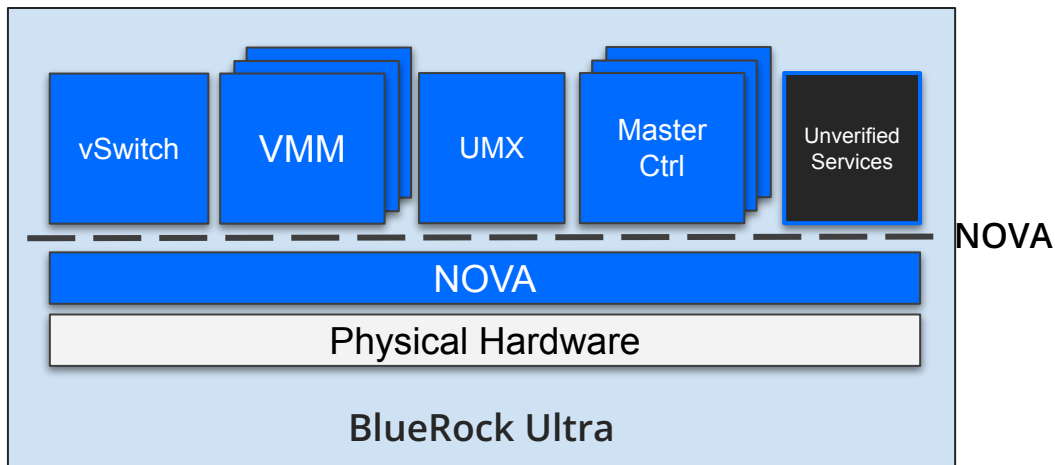
Read more: <https://bluerock.io/wp-content/uploads/2024/04/nova-interface.pdf>

Case Studies

BlueRock ULTRA™

A range of verifications within the ULTRA platform.

- BSON parser
- Concurrency: Semaphore, Mutex, Client_lock
- Applications (partial)
 - Virtual Machine Monitor
 - Virtual Network Switch (VIRTIO)
 - Console Multiplexor
 - Device Drivers (serial & DMA w/ interrupts)
- NOVA (partial, in progress)



Separation across the Stack

Definition semaphoreI (o : option ghost) : mpred :=

match o with

| None $\Rightarrow$ emp

| Some $\gamma \Rightarrow$

rinv semaphore.NS γ .(lg_inv)

(Exists (c : Z) (** counter value *)

(w : N) (** # of pending downs *)

(r : N) (** # of pending ups *)

(sv : N) (** value of NOVA semaphore

qu,

_field "Sys::Semaphore::_count" $\rightarrow$ atomicR Ti₆₄ c 1 **

pureR (

sm.value γ .(sm_sm) sv ** sm.queue γ .(sm_sm) qu **

releaserF γ sv ** waiterF γ (N.of_nat (length qu)) **

waiterA γ w ** releaserA γ r **

can_upF γ w **

boundA γ c **

qftickets.auth γ .(lg_tickets) ((r - sv) + w) **

valueA γ (Z.to_N (Z.of_N w + c))%N ** [| (w + Z.min 0 c = r)%Z |])

end.

(Relocatable)
Iris concurrency invariant

C++ state.

OS State

Ghost state for bookkeeping

Fair Semaphores

The Behavior

State – reflected as separation logic resources

- A value (N)

`sm.value g n`

- A queue of blocked E

`sm.queue g ls`

NOVA
choice.

```
letI* g := resolve_sm ec.pd {DN} sel Q in
letI* _ := sm_wait g ec in
    sm.acquire z g ec Q
  /\ sm.timeout time g ec Q
wp_hypercall ec ctrl_sm(1, sel, z, time) Q
```

4.4.5 Control Semaphore

Parameters:

```
status = ctrl_sm (SEL_obj sm,      // Semaphore
                  UINT ticks);    // Deadline Timeout
```

Flags:

0	0	Z	D
3	2	1	0

Does not

refer to a SM Object Capability ($CAP_{OBJ_{SM}}$) with

refer to a SM Object Capability ($CAP_{OBJ_{SM}}$) with

semaphore, then the microhypervisor has released one
microhypervisor has incremented the semaphore counter. The
ignored.

microhypervisor has
microhypervisor has
 $C_{CURRENT}$ unblocks

} sel Q in

(0, sel, _)

Hypercalls expressed
with "big-step" predicate
transformers using Iris
atomic updates.

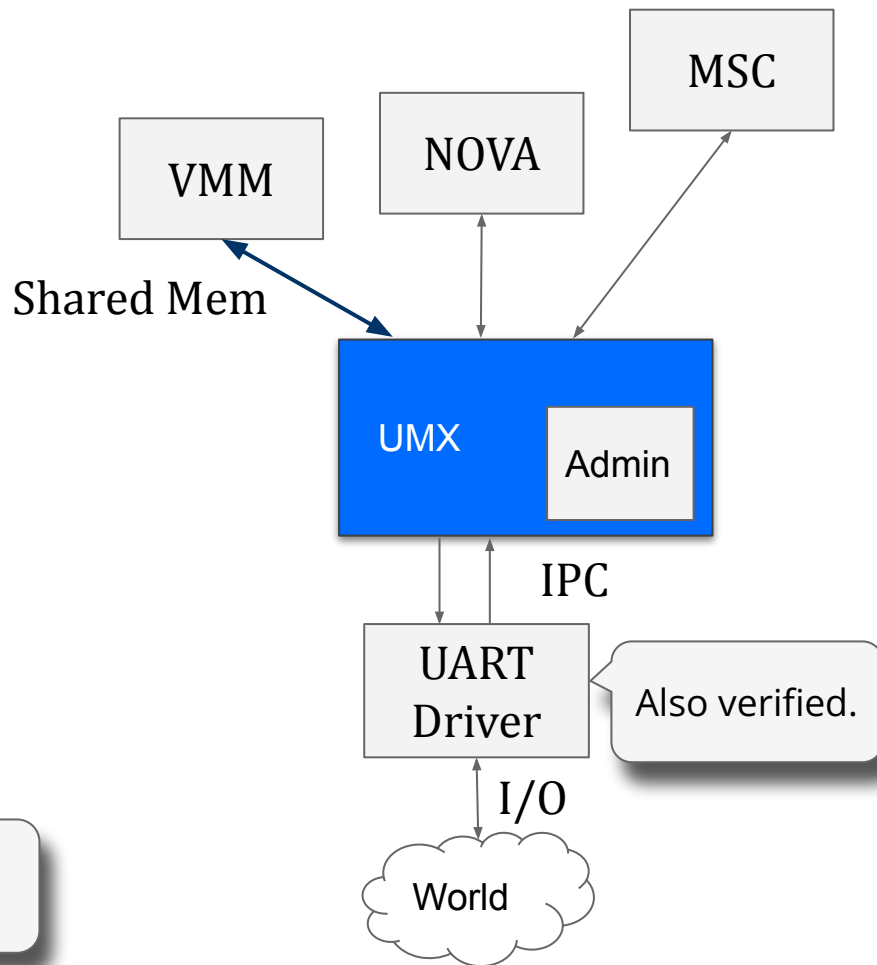
or that capability

CPU than the CPU

The BHV Terminal Multiplexer (UART Multiplexer)

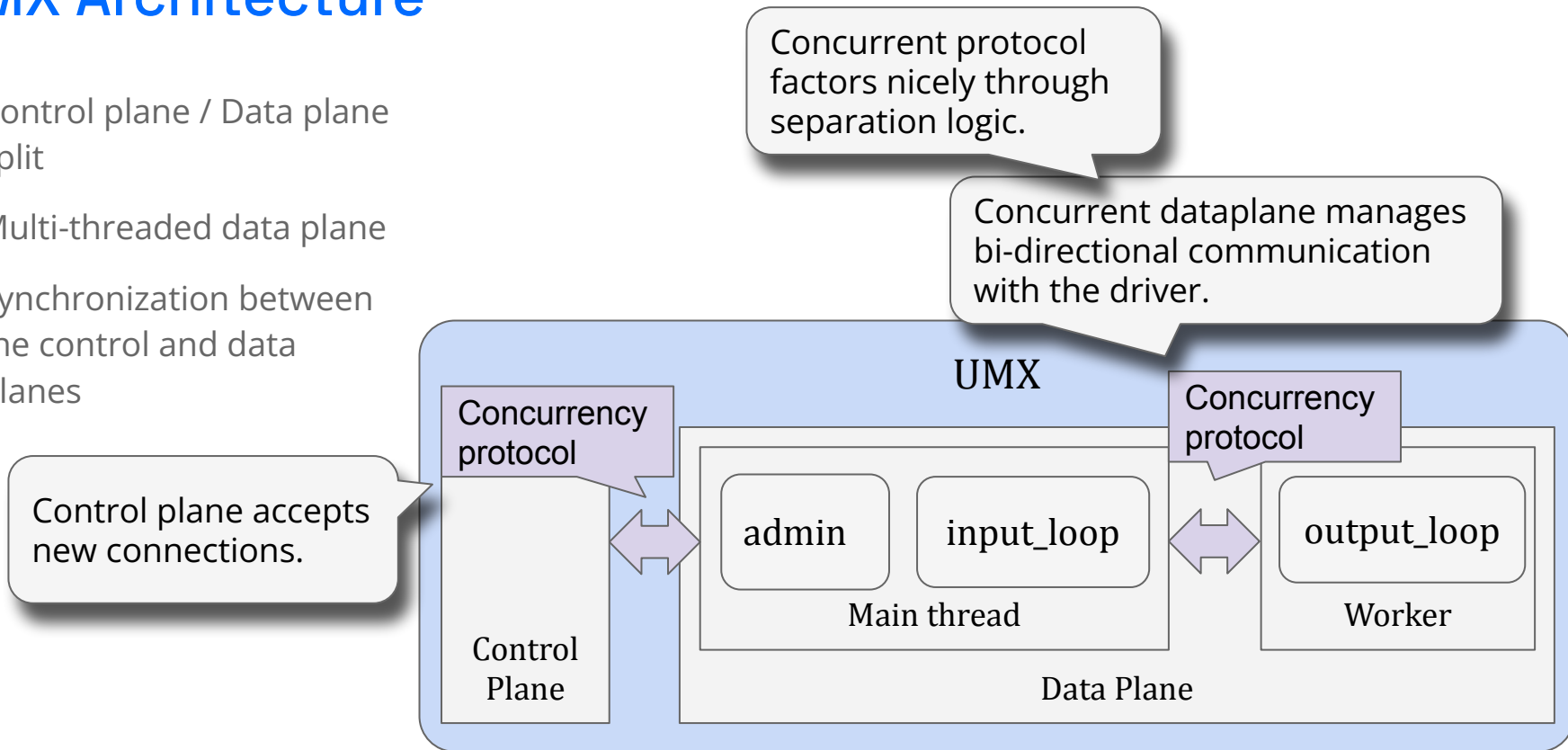
```
Welcome to the BHV™ UART multiplexer console
BHV> list
active clients:
0: msc
1: nova
2: logger
3: vmm (hydra)
4: vsmx
5: policy-manager
6: VMI (ogre)
7: vmm (ogre)
8: VMI (hydra)
9: vswitch
10: ogre virtio console
11: hydra virtio console
BHV> █
```

Verified code running on the ULTRA console



UMX Architecture

- Control plane / Data plane split
- Multi-threaded data plane
- Synchronization between the control and data planes



Virtual Network Switch

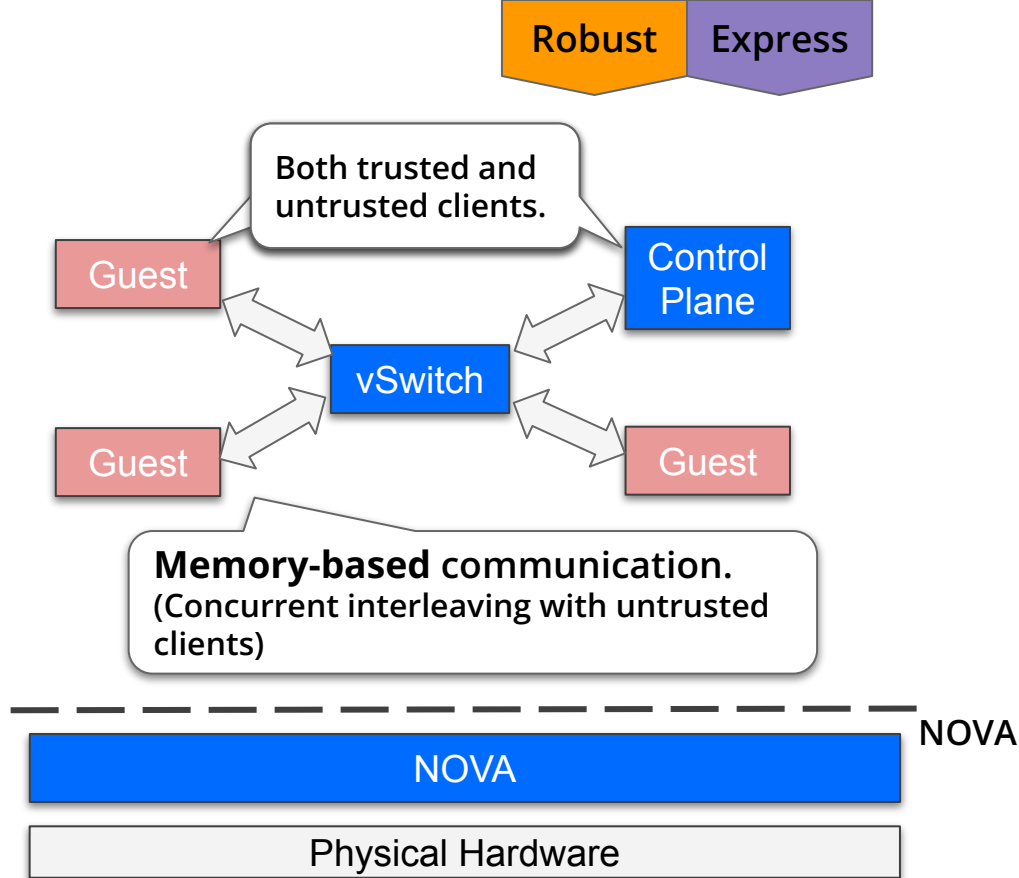
HASH SBIR

Understandable Property

- Acts like a layer 2 switch.

Implementation

- Both trusted and untrusted clients.
- Communicates via (standard) VIRTIO-net protocol.
 - In-memory queues.
 - Potential for data races.
- Concurrent implementation.



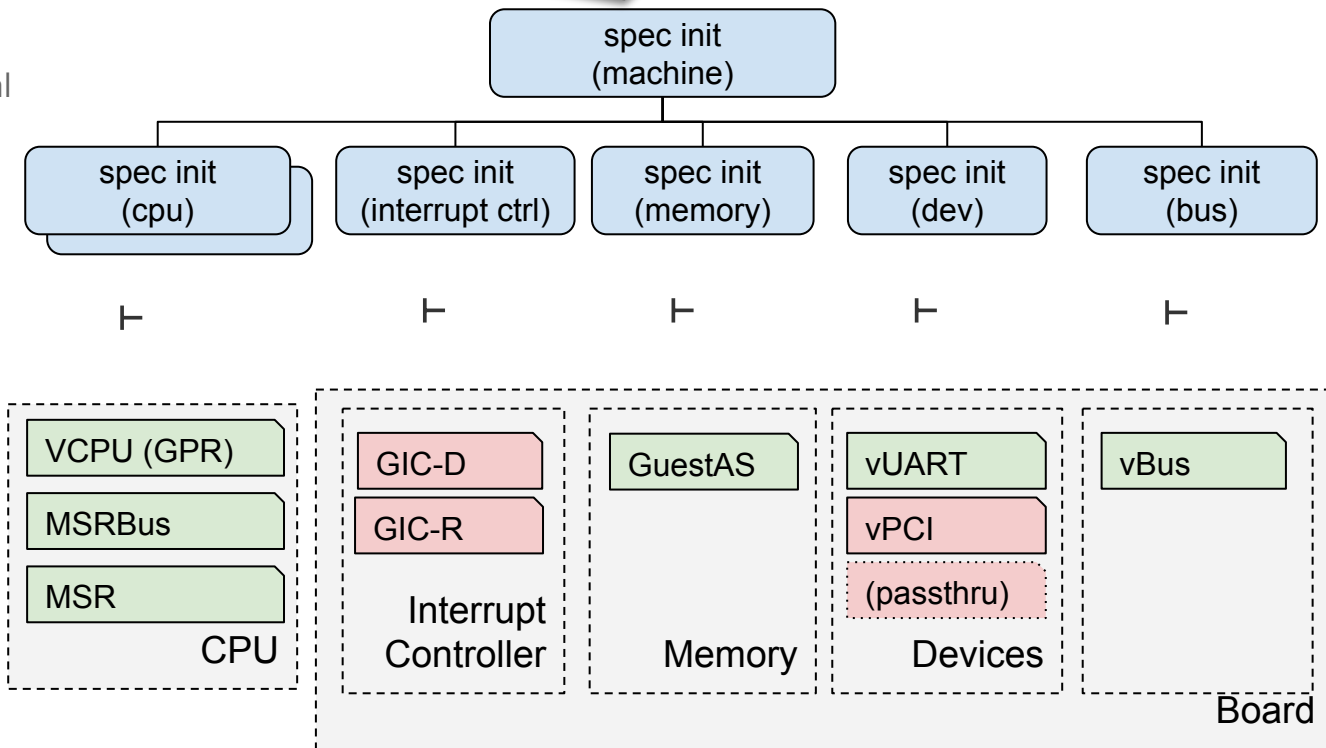
Virtual Machine Monitor

Decomposition supported
by **Spectra library**.

Modular refinement
verification for process-calculi
specifications.

Separation logic enables a natural
proof architecture.

- Separate components maintain separate simulations.
- Communication between components.
- Simulation connects user-mode C++ code with the NOVA specification for hardware virtualization.





Next Steps

Commercialization & Collaboration

Commercial:

- Category Labs – blockchain company
- SkyLabs AI – "sister company" working on AI-assisted verification
- Riverside Research (upcoming) – Rust front-end

Collaborations:

- Yale – BRiCk foundations
- UIC (potential) – concurrent library verification

Expanding Commercialization

Relevance to PROVERS performers?

- What aspects of the verification process do users find important?
- What are the major stumbling blocks for new users?
 - Are there effective ways to streamline the on-ramp?
- What verification models are being explored?

Verification targets:

- Are performers interested in C++?
 - What features are of particular interest? Exceptions? Standard library? Etc.
- Properties of interest?
 - Behavioral refinements?
 - Safety properties?

Is there any opportunity to interact with PROVERS performers, especially users, to understand their pain points?
Are there any relevant reports that we should look at?